

AUFGABE ZUR VORLESUNG

Übung 06 – Secrets

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Docker: Passwörter richtig behandeln
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Vorlesung: Deck *docker-compose* **Ziel:** Verstehen, warum Passwörter in Umgebungsvariablen problematisch sind, und sie stattdessen als **Secret** behandeln.

Kontext

Bisher stand das DB-Passwort im Klartext in der `compose.yaml` bzw. als Umgebungsvariable. Das Problem: Umgebungsvariablen sind **leicht auslesbar** – über `docker inspect`, in Logs, in der Prozessliste. Für ein Übungsprojekt ist das egal, in echten Systemen ist es ein ernstes Sicherheitsrisiko.

Docker Compose kennt dafür **file-based secrets**: Das Passwort liegt in einer eigenen Datei und wird zur Laufzeit unter `/run/secrets/<name>` in den Container eingeblendet. PostgreSQL (`POSTGRES_PASSWORD_FILE`) und unser `log_tool` (`DB_PASSWORD_FILE`) können beide aus einer solchen Datei lesen.

Aufgabenstellung

1. **Zeigen Sie das Leck:** Starten Sie kurz einen Container mit einem Passwort in einer Umgebungsvariablen und lesen Sie es mit `docker inspect` wieder aus.
2. Legen Sie die Secret-Datei an: `db_password.txt` (aus der `.example`-Vorlage). Warum darf diese Datei **nicht** ins Git-Repo?
3. Sehen Sie sich die `compose.yaml` an: Wo wird das Secret definiert, wo benutzt? Starten Sie den Stack.
4. Fügen Sie mit `log_tool` Einträge hinzu und listen Sie sie – es funktioniert genauso wie vorher, aber ohne Passwort im Klartext in der Konfiguration.
5. Weisen Sie nach, dass in der `compose.yaml` **kein** Passwort mehr steht und `docker inspect` beim `log_tool`-Dienst keines mehr als Umgebungsvariable zeigt.

Tipp: In `.gitignore` dieses Repos ist `**/db_password.txt` ausgeschlossen – committet wird nur die harmlose `db_password.txt.example`.

Musterlösung

```
cd uebungen/06_secrets
```

```
# 1) Das Leck vorführen: Passwort per Env -> per inspect wieder sichtbar.  
# (Ein simpler alpine-Container genügt; er läuft ruhig vor sich hin.)  
docker run -d --name leak -e DB_PASSWORD=streng-geheim alpine sleep 300  
docker inspect leak --format '{{range .Config.Env}}{{println .}}{{end}}' | grep  
DB_PASSWORD
```

```
# -> DB_PASSWORD=streng-geheim (im Klartext auslesbar!)
docker rm -f leak

# 2) Secret-Datei aus der Vorlage anlegen (NICHT ins Git!)
cp db_password.txt.example db_password.txt

# 3) Stack mit Secret starten
docker compose up -d
docker compose ps

# 4) log_tool nutzt das Secret aus /run/secrets/db_password
docker compose run --rm log_tool add "Jetzt mit Secret"
docker compose run --rm log_tool list

# 5) Nachweis: kein Passwort in der Konfiguration / in den Env-Variablen
grep -i password compose.yaml # nur *_PASSWORD_FILE, kein Klartext
docker inspect $(docker compose ps -q db) \
  --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -i password
# -> nur POSTGRES_PASSWORD_FILE=/run/secrets/db_password (kein Klartext)

docker compose down
```

Bonus / Ausblick: Secrets in Docker Swarm

Compose-Secrets sind Dateien. In einem Produktions-Cluster mit **Docker Swarm** gibt es einen echten, verschlüsselten Secret-Speicher:

```
# (nur zum Ansehen – erfordert einen aktiven Swarm)
docker swarm init
printf 'streng-geheim' | docker secret create db_password -
docker secret ls
```

Das Secret wird im Swarm verschlüsselt abgelegt und nur den Diensten zugänglich gemacht, die es brauchen. Mehr dazu in Übung 08.

Was Sie gelernt haben: Passwörter in Umgebungsvariablen lecken über `docker inspect`; Compose-Secrets legen sie in eine separate Datei und blenden sie unter `/run/secrets/` ein; solche Dateien gehören nie ins Repository.